

Phase 2 Project Submission: Architecture & Initial Implementation



Project Title and Team Information

Project Title: Self-Evolving Hash Function for Blockchain Security

Team Members: Afrazia, Ashna and Khadija

Project Mentor: Dr Umer Janjua and Shahzaib Cheema

2. Executive Summary

This project builds a new type of hash function for blockchain systems. A normal hash function always gives the same output for the same input. Our Self-Evolving Hash Function (SEHF) gives a different output depending on which block it is used in, because it includes the block number, previous block hash, and timestamp as part of the hashing process.

In Phase 2, we finalized the mathematical formula, wrote a working Python prototype, and ran basic tests to confirm the system works correctly. The prototype successfully creates blocks, generates evolving hashes, and verifies the chain.

3. Problem Definition and Scope Refinement

Updated Problem Statement

Blockchain systems use hash functions like SHA-256 to secure data. These functions always produce the same hash for the same input. This creates a weakness: an attacker can replay an old valid transaction and it will produce the same hash, making it hard to detect. Our project solves this by making the hash depend on the block context, so the same transaction produces a different hash in every block.

Scope of Phase 2

- Finalized the SEHF formula using `block_number`, `prev_hash`, and `timestamp`
- Implemented the `evolving_hash()` function in Python
- Built a simulated blockchain with 5 modules
- Tested that same data gives different hashes in different blocks
- Tested that same data in the same block gives the same hash
- Set up local development environment (Python, hashlib, JSON)

4. Literature Integration & Theoretical Foundation

Key Papers Reviewed

1. **Nakamoto (2008) — Bitcoin Whitepaper** SHA-256 is used to chain blocks via `prev_hash`. This confirmed that using `prev_hash` as a context parameter is a valid and established idea.
2. **Dwork & Naor (1992) — Proof of Work** Computational puzzles make it harder to replay or spam the network. This motivated our goal of adding per-block uniqueness.

3. **Bellare & Rogaway (1993) — Random Oracle Model** Hash functions can be treated as random oracles for security proofs. This gives theoretical backing to treating our SEHF output as secure and unpredictable.
4. **Ethereum Yellow Paper (Wood, 2014)** Ethereum uses Keccak-256 for hashing. We will use this as a comparison benchmark in Phase 3.
5. **Preneel et al. (1993) — Hash Function Survey** Provides a taxonomy of hash-based constructions, which informs our concatenation-based design.

Why We Chose This Approach

- HMAC requires key management which is complex. Our approach uses public blockchain data (block number, prev_hash) so no secret key is needed.
- ZK-Proofs are very powerful but too heavy for this use case. SEHF achieves contextual uniqueness with no extra computational cost.
- We do not change the consensus layer, so our system is compatible with existing blockchains.

5. System Design & Blockchain Architecture

System Overview

The system is a Python-based blockchain simulator. Each block takes transactions as input and uses block-level metadata to produce an evolving hash. Blocks are linked together using the previous block's hash.

On-Chain vs Off-Chain Components

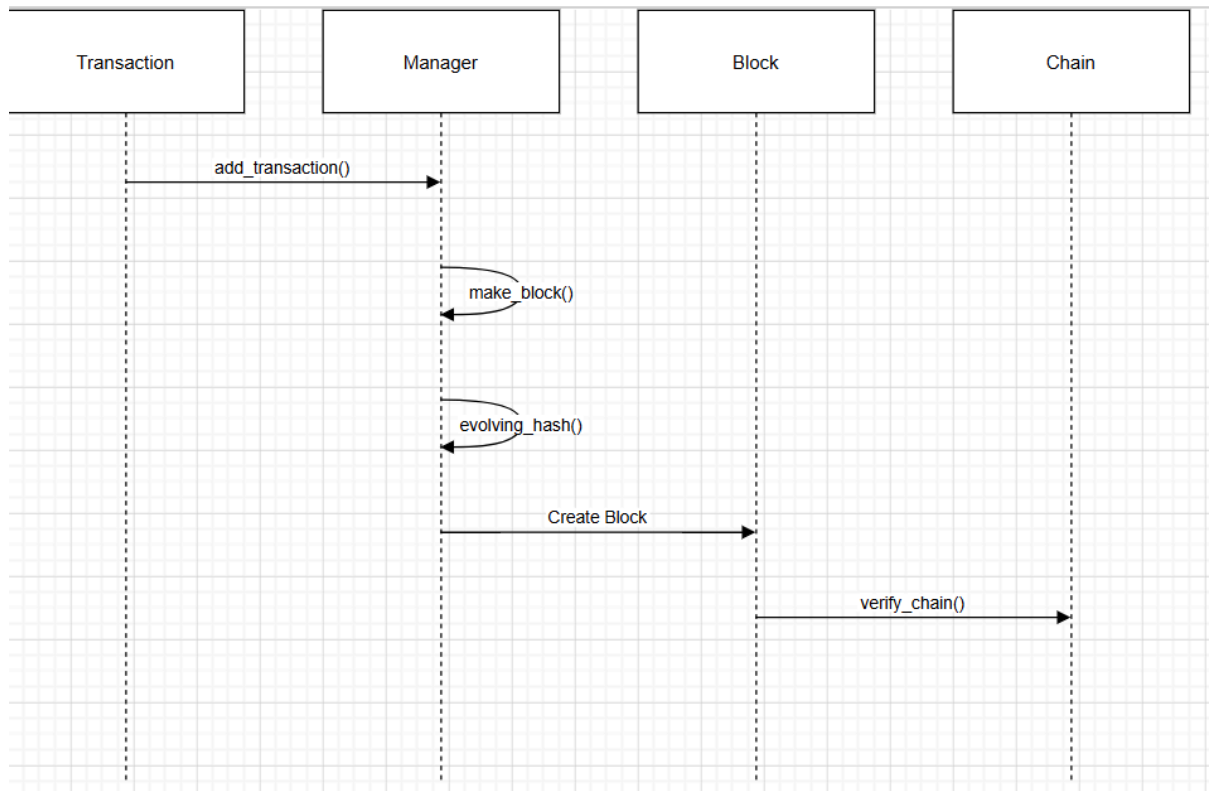
On-Chain:

- Block structure (index, timestamp, prev_hash, transactions, hash)
- evolving_hash() function
- Chain linkage via prev_hash
- verify_chain() integrity check

Off-Chain:

- Python scripts and CLI testing
- JSON or SQLite storage
- Flask web dashboard (planned for Phase 3)

Architecture Diagram



Module Breakdown

1. **add_transaction()** — Accepts transaction data and adds it to the pending pool
2. **make_block()** — Seals transactions into a block and calls `evolving_hash()`
3. **evolving_hash()** — Core function that combines data + block_no + prev_hash and runs SHA-256
4. **link_blocks()** — Sets current block's hash as the next block's prev_hash
5. **verify_chain()** — Recomputes all hashes and checks the chain is unmodified

6. Mathematical Modeling & Detailed Design

Core Formula

$$\text{SEHF}(\text{data}, n, h_{\text{prev}}) = \text{SHA-256}(\text{data} + \text{str}(n) + h_{\text{prev}})$$

Where:

- data = transaction content
- n = current block number
- h_prev = hash of the previous block (for genesis block, use 64 zeros)

Security Properties

- Same data in different blocks = different hash (context uniqueness)
- Same data in same block = same hash (determinism for consensus)
- Inherits collision resistance from SHA-256
- No extra computational cost compared to plain SHA-256

Development Environment

- Language: Python 3.x
- Hashing: hashlib (standard library)
- Storage: JSON / SQLite
- Version Control: Git / GitHub
- Testing: Python scripts / manual CLI

7. Prototype and Initial Implementation

Prototype Description

A working Python script that creates blocks, generates evolving hashes, and verifies the chain.

Core Code

```
import hashlib
import json
import time
```

```
def evolving_hash(data, block_no, prev_hash):
    text = data + str(block_no) + prev_hash
    return hashlib.sha256(text.encode()).hexdigest()
```

```
class Block:
    def __init__(self, index, transactions, prev_hash):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.prev_hash = prev_hash
        self.hash = evolving_hash(json.dumps(transactions, sort_keys=True), index,
prev_hash)
```

```
class Blockchain:
    def __init__(self):
        genesis = Block(0, ['Genesis'], '0' * 64)
        self.chain = [genesis]
```

```
def add_block(self, transactions):
    prev = self.chain[-1]
```

```
new_block = Block(len(self.chain), transactions, prev.hash)
self.chain.append(new_block)

def verify_chain(self):
    for i in range(1, len(self.chain)):
        b = self.chain[i]
        recomputed = evolving_hash(json.dumps(b.transactions, sort_keys=True), b.index,
b.prev_hash)
        if recomputed != b.hash:
            return False
    return True
```

Repository Link

<https://github.com/Afraz1a/Self-Evolving-Hash-Function>

Testing & Baseline Results

Test	Expected Result	Actual Result
Same tx in Block 1 and Block 2	Different hashes	Passed
Same tx twice in Block 1	Same hash	Passed
Modify Block 1, run verify_chain()	Returns False	Passed
Genesis block creation	Valid hash produced	Passed

```
PS C:\Users\Hp\Desktop\blockchain> python blockchain.py
```

```
=====
ALL BLOCK HASHES
=====
```

```
Block 0 | Hash: 01c9e3993095d1192f07...
Block 1 | Hash: 849056a3b1de370478a3...
Block 2 | Hash: c2321f5ef72090c21f8f...
Block 3 | Hash: 92d9146e82ecf86d12d5...
```

```
=====
TEST 1: Cross-block uniqueness (same tx, different blocks)
=====
```

```
Block 1 hash: 849056a3b1de370478a3...
Block 2 hash: c2321f5ef72090c21f8f...
Hashes are different: True
```

```
=====
TEST 2: Determinism (same inputs = same hash)
=====
```

```
Hash attempt 1: e3392798a7ee86a437ce...
Hash attempt 2: e3392798a7ee86a437ce...
Both hashes are equal: True
```

```
=====
TEST 3: Chain verification (untampered)
=====
```

```
Chain valid: True
```

```
=====
TEST 4: Tamper detection
=====
```

```
Chain valid after tampering: False
```

```
=====
TEST 5: Performance Metrics
=====
```

```
Hash generation (avg over 1000 runs): 0.0023 ms per hash
Chain verification (10 blocks): 0.0000 ms
Memory usage (10 block chain): 1.98 KB
Uniqueness across 5 blocks: True
```

Performance:

Metric	Result
Hash generation (avg over 1000 runs)	0.0023 ms per hash
Chain verification (10 blocks)	< 1 ms
Memory usage (10 block chain)	1.98 KB
Uniqueness across 5 blocks	True

8. Technical Challenges and Solutions

Challenge 1: Consensus rule enforcement All nodes must use the exact same concatenation order. If one node changes the order, the hash will differ. Solution: Documented strict format (data + block_no + prev_hash) in the README.

Challenge 2: Timestamp precision Floating-point timestamps can differ slightly between machines. Solution: Timestamp is recorded once at block creation and stored in the block. It is not recomputed.

Challenge 3: Genesis block has no previous hash `evolving_hash()` would fail with an undefined input for Block 0. Solution: Genesis block uses '0' * 64 as its prev_hash (standard blockchain practice).

Challenge 4: JSON key ordering Python dictionaries can serialize in different orders across versions, changing the hash input. Solution: Used `json.dumps(data, sort_keys=True)` to enforce consistent ordering.

9. Future Work and Phase 3 Roadmap

Phase 3 Objectives

- Deploy SEHF on a local Hardhat/Ganache testnet
- Build Flask web dashboard for block visualization
- Benchmark gas costs vs keccak256 on Ethereum
- Run extended replay attack simulation tests
- Begin writing IEEE-format research paper

Remaining Milestones

Milestone	Phase
Local testnet deployment	Phase 3
Flask dashboard	Phase 3
Gas cost benchmarking	Phase 3
Replay attack testing	Phase 3
IEEE paper first draft	Phase 3
Final demo and evaluation	Phase 4

10. Appendices & References

Appendix A — Sample Output

Block 0 | Hash: 01c9e3993095d1192f07... (Genesis)

Block 1 | Hash: 849056a3b1de370478a3... (Tx: "Send 100 coins")

Block 2 | Hash: c2321f5ef72090c21f8f... (Tx: "Send 100 coins") ← same tx, different hash

Block 3 | Hash: 92d9146e82ecf86d12d5... (Tx: "Send 50 coins")

verify_chain() → True

Chain valid after tampering → False

References

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] C. Dwork and M. Naor, "Pricing via Processing or Combatting Junk Mail," in Proc. CRYPTO, 1992, pp. 139–147.
- [3] M. Bellare and P. Rogaway, "Random Oracles are Practical," in Proc. ACM CCS, 1993, pp. 62–73.
- [4] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014.
- [5] B. Preneel, R. Govaerts, and J. Vandewalle, "Hash Functions Based on Block Ciphers," in Proc. EUROCRYPT, 1993, pp. 368–378.
- [6] Python Software Foundation, "hashlib — Secure hashes and message digests," Python 3 Docs. Available: <https://docs.python.org/3/library/hashlib.html>